

CS4231
Parallel and Distributed Algorithms

Lecture 9
Instructor: Haifeng YU

Review of Last Lecture

<u>Failure Model and Timing Model</u>	<u>Consensus Protocol</u>
Ver 0: No node or link failures	Trivial – all-to-all broadcast
Ver 1: Node crash failures; Channels are reliable; Synchronous;	$(f+1)$ -round protocol can tolerate f crash failures
Ver 2: No node failures; Channels may drop messages (the coordinated attack problem)	Impossible without error Randomized algorithm with $1/r$ error prob
Ver 3: Node crash failures; Channels are reliable; Asynchronous;	This lecture
Ver 4: Node Byzantine failures; Channels are reliable; Synchronous; (the Byzantine Generals problem)	This lecture

Today's Roadmap

- “Agreement”
 - Also called consensus
- Ver 3: Node crash failures; Channels are reliable; Asynchronous;
- Ver 4: Node Byzantine failures; Channels are reliable; Synchronous; (the Byzantine Generals problem)

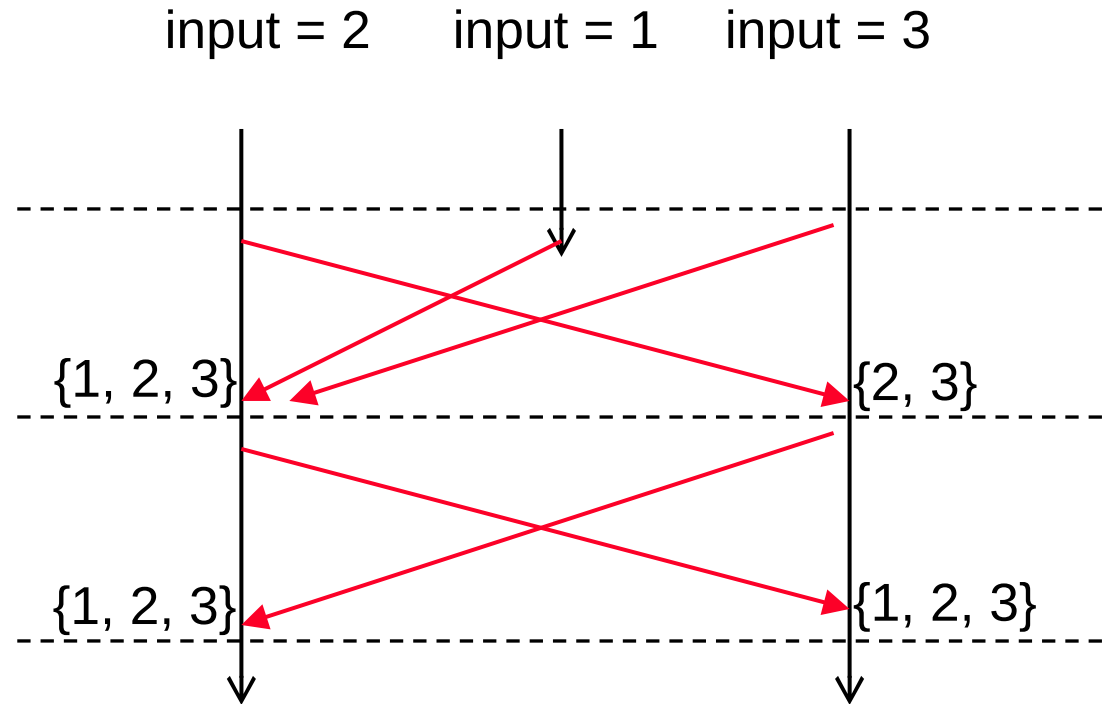
Distributed Consensus Version 3: Consensus with Node Crash Failures/Asynchronous

- Failure model:
 - Nodes may experience **crash failures**
 - Communication channels are reliable
- Timing model:
 - **Asynchronous**: Process delay and message delay are **finite but unbounded**
- The delay of each message is finite, but you cannot find a bound such that all message delays are below that bound
 - In practice, there can be messages delayed for a long time
- We can no longer define a round
 - If we don't receive a message for a long time, we don't know if the sender has failed or the message is just delayed

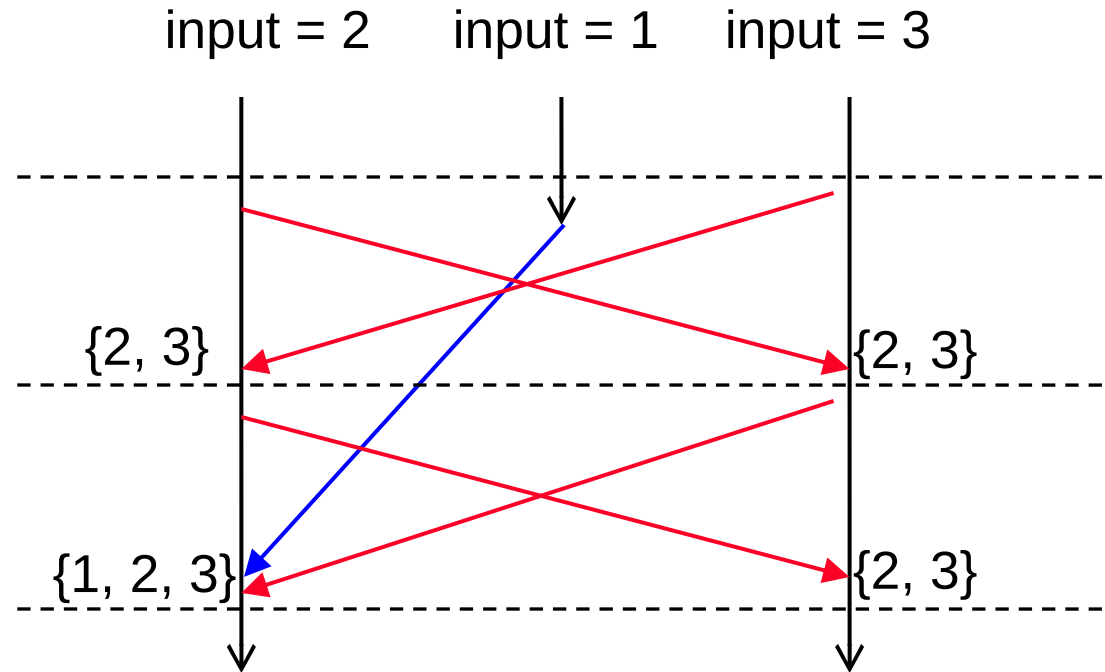
Distributed Consensus Version 3: Consensus with Node Crash Failures/Asynchronous

- Goal:
 - **Termination**: All nodes (that have not failed) eventually decide
 - **Agreement**: All nodes that decide should decide on the same value
 - **Validity**: If all nodes have the same initial input, that value should be the only possible decision value. Otherwise nodes are allowed to decide on anything (except that they still need to satisfy the Agreement requirement)

Distributed Consensus Version 3: How does the round-based protocol fail



Distributed Consensus Version 3: How does the round-based protocol fail



Will using 3 rounds solve the problem? Why?

Quick Poll <https://pollev.com/haifengyu229>

Distributed Consensus Version 3: The FLP Impossibility Theorem

- FLP Theorem [Fischer,Lynch,Paterson'85]:
 - The distributed consensus problem under the asynchronous communication model is impossible to solve even with a single node crash failure
 - Arguably **the most fundamental result in distributed computing** so far
- Fundamental reason:
 - The protocol is unable to accurately detect node failure

Formalisms for FLP Theorem

- **Goal: Abstract the execution of any possible deterministic protocol**
- Each process has some local state and two special variables
 - $\text{input} \in \{0, 1\}$ and $\text{decision} \in \{\text{null}, 0, 1\}$
 - decision is initially null, and can be written exactly once
- Each communication channel has some state:
 - Messages “on-the-fly”
- The message system captures the state of all communication channels
 - $\{(p, m) \mid \text{message } m \text{ is on the fly to process } p\}$
 - All messages are distinct
- Send = add (dest, content) to the message system
- Receive (when invoked by process p) =
 - Remove some $(p, \text{content})$ from message system and then return content, OR
 - Leave the message system unchanged and return null
 - Out-of-order or FIFO?
 - Non-blocking receive or blocking receive?

Formalisms for FLP Theorem

- Global state of the system include all process states and message system state
 - A deterministic state machine
- A **step** of in a protocol takes the system from one global state to another:
 - By executing the following on process p
 - receive a message m (m can be null);
 - based on p 's local state and m , send an arbitrary but finite number of messages
 - based on p 's local state and m , change p 's local state to some new state
- Given a global state, each step is fully described by p 's receiving m
 - Call (p, m) as an **event**
 - **Events are inputs to the state machine that cause state transitions**
 - An event e **can be applied** to global state G if either m is null or (p, m) is in the message system

Formalisms for FLP Theorem

- The “execution” of any protocol can be abstracted to be an infinite sequence of events
 - Each “execution” may be different though
 - Can always make a protocol not to terminate
 - Each process must be able to handle null messages
 - Decisions are made when the decision variable is set
 - This abstraction is necessary to properly define failed (faulty) processes
- A schedule σ is a sequence of events that captures the execution of some protocol
 - σ **can be applied** to G if the events can be applied to G in the order in σ
 - $G' = \sigma(G)$ means that if we apply σ to G , we will end up with G'
 - Need to be careful when we write $\sigma(G)$, since σ may or may not be applied to G

Formalisms for FLP Theorem

- Given a consensus protocol A, a global state G2 is **reachable** from G1 if there is a schedule σ (of A) such $G2 = \sigma(G1)$.
- By requirements of consensus, the protocol A must satisfy
 - **Agreement**: No reachable global state from any initial state has more than one decision.
 - **Validity**: If all nodes have the same initial input, they should all decide on that
 - **Termination**: Eventually all processes decide

Formalisms for Asynchronous System and Failures

- Abstracting asynchronous systems
- **Processes have unbounded but finite delay:**
 - A nonfaulty process takes infinite number of steps.
 - A faulty process takes a finite number of steps.
 - If we consider only finite sequences, then we cannot distinguish faulty from nonfaulty processes
- **Messages have unbounded but finite delay:**
 - Every message is eventually delivered
 - If there is a message (p, m) in the message system and p invokes `receive()` multiple times, then the message system can only return null finite number of times
- At most one faulty process

Proof for FLP Theorem

- An extremely beautiful but hard proof
 - Perhaps the hardest proof in this course
- General proof technique:
 - We will act as the adversary to defeat the consensus protocol
 - We (scheduler) can pick **which messages to deliver** and **which process will take the next step** (under the constraints of asynchronous system)
 - Our goal is to prevent the protocol from ever deciding
- Classification of global states
 - G is **0-valent** if 0 is the only possible decision reachable from G
Processes in G may or may not yet decided on 0, but if not, they will eventually decide on 0
 - G is **1-valent** if 1 is the only possible decision reachable from G
 - G is **univalent** if G is either 0-valent or 1-valent
 - G is **bivalent** if it is not univalent

Proof for FLP Theorem

- We will prove that we (the adversary) can always keep the system in a bivalent state
- **Lemma 1: For any protocol A, there exists a bivalent initial state.**
 - Prove by contradiction and consider $n+1$ initial states with input vector being $(0,0,\dots, 0)$, $(1, 0, \dots, 0)$, $(1, 1, 0, \dots, 0)$, ..., $(1, 1, \dots, 1)$

0-valent

$(0, 0, 0, 0)$

$(1, 0, 0, 0)$

$(1, 1, 0, 0)$

$(1, 1, 1, 0)$

1-valent

$(1, 1, 1, 1)$

- There must be two adjacent initial states S_0 and S_1 where S_0 is 0-valent and S_1 is 1-valent. (We are assuming no bivalent state.)
- S_0 and S_1 differ by the input to a single process p .
- Consider an execution starting from S_0 where p fails at the very beginning. If the decision is 1, then S_0 is not 0-valent. If the decision is 0, then S_1 is not 1-valent. Contradiction.

Proof for FLP Theorem

- Lemma 2: Let σ_1 and σ_2 be two schedules such that the set of processes executing steps in σ_1 are disjoint from the set that execute steps in σ_2 . Then for any G that σ_1 and σ_2 can both be applied, we have $\sigma_1(\sigma_2(G)) = \sigma_2(\sigma_1(G))$.
 - Proof by induction on $k = \max(|\sigma_1|, |\sigma_2|)$
- Induction base $k = 1$: $e_1(e_2(G)) = e_2(e_1(G))$
 - Suppose $e_1 = (p_1, m_1)$ and $e_2 = (p_2, m_2)$. Since e_1 can be applied to G , it means either m_1 is null or (p_1, m_1) is in the message system. The same is for e_2 . Because $p_1 \neq p_2$, e_1 can be applied to $e_2(G)$ and e_2 can be applied to $e_1(G)$.
 - Let $G_1 = e_1(e_2(G))$ and $G_2 = e_2(e_1(G))$. Then the state of the message system is the same in G_1 as in G_2 . The states of all processes are the same in G_1 and G_2 as well. Thus $G_1 = G_2$.

Proof for FLP Theorem

- Lemma 2: Let σ_1 and σ_2 be two schedules such that the set of processes executing steps in σ_1 are disjoint from the set that execute steps in σ_2 . Then for any G that σ_1 and σ_2 can both be applied, we have $\sigma_1(\sigma_2(G)) = \sigma_2(\sigma_1(G))$.
 - Proof by induction on $k = \max(|\sigma_1|, |\sigma_2|)$
 - First, verify that $\sigma_1(\sigma_2(G))$ and $\sigma_2(\sigma_1(G))$ are both well-defined
- Induction step for $k+1$:
 - Case 1: $|\sigma_1| = k+1$ and $|\sigma_2| \leq k$
Suppose the first event in σ_1 is e and $\sigma_1 = (\sigma|e)$ where $|\sigma| = k$. Then $\sigma_1(\sigma_2(G)) = \sigma(e(\sigma_2(G))) = \sigma(\sigma_2(e(G))) = \sigma_2(\sigma(e(G))) = \sigma_2(\sigma_1(G))$
 - Case 2: $|\sigma_1| \leq k$ and $|\sigma_2| = k+1$. Same as case 1
 - Case 3: $|\sigma_1| = k+1$ and $|\sigma_2| = k+1$
Suppose the first event in σ_2 is e and $\sigma_2 = (\sigma|e)$ where $|\sigma| = k$. Then $\sigma_1(\sigma_2(G)) = \sigma_1(\sigma(e(G))) = \sigma(\sigma_1(e(G))) = \sigma(e(\sigma_1(G))) = \sigma_2(\sigma_1(G))$. (Notice that we use case 1 in the proof.)

Proof for FLP Theorem

- Lemma 3: Let G be a global state, and $e = (p, m)$ is an event that can be applied to G . Let W be the set of global states that is reachable from G without applying e , then e can be applied to any state in W .
 - Proof is trivial.
- Lemma 4: Let G be a bivalent state, and $e = (p, m)$ is any event that can be applied to G . Let W be the set of global states that is reachable from G (G is in W) without applying e , and $V = e(W)$ to be the set of global states by applying e to the states in W . Then V contains a bivalent state.
 - We will prove later.

Proof for FLP Theorem

- Proof for FLP Theorem:
 - We act as the scheduler, and start with some initial bivalent state (by Lemma 1)
 - Processes take steps in round-robin fashion. Imagine that it is process p 's turn.
 - If the message system contain no messages for p , let $e = (p, \text{null})$.
 - Otherwise consider the oldest message m destined to p , and let $e = (p, m)$.
 - Let G be the current state.
 - Execute (p, m) if $e(G)$ is bivalent (how to determine bivalency?).
 - Otherwise find (how?) a finite length σ that does not contain e and $e(\sigma(G))$ is bivalent (by Lemma 4).
 - Apply σ and then apply e .
 - The system will always be in a bivalent state.

Proof for FLP Theorem

- The scheduler plays by rules:
 - All nonfaulty processes takes infinite number of steps
 - All messages are eventually delivered
 - Process delays and message delays may not be bounded (why? and why is this OK?)
- If process delays and message delays are bounded, then consensus is solvable.

Implications of FLP Theorem

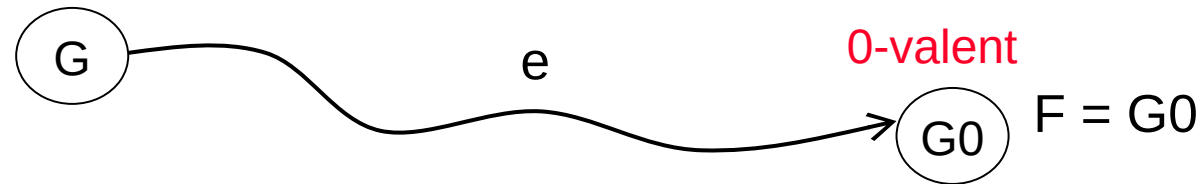
- Complete correctness if not possible
- In practice, we may live with very low probability of disagreement
- In practice, we may live with very low probability of blocking (non-termination)
 - Two-phase commit or even three-phase commit can block forever
- Randomization

Proof for Lemma 4

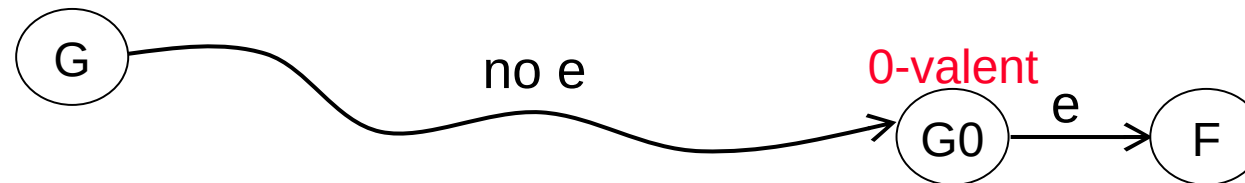
- Lemma 4: Let G be a bivalent state, and $e = (p, m)$ is any event that can be applied to G . Let W be the set of global states that is reachable from G (G is in W) without applying e , and $V = e(W)$ to be the set of global states by applying e to the states in W . Then V contains a bivalent state.
 - Prove by contradiction and assume that V does not.
 - This assumption is always carried along when proving the next 4 claims.

Proof for Lemma 4

- Claim 1: There must exist some schedule σ such that σ contains the event e and $\sigma(G)$ is 0-valent
 - Proof: G is bivalent thus we must have a 0-valent state G_0 reachable from G where $G_0 = \sigma_1(G)$. Now consider two cases.
 - Case 1: σ_1 contains event e . Here we will let $\sigma = \sigma_1$. We are done.



- Case 2: σ_1 does not contain event e . We let $\sigma = e|\sigma_1$, and hence $\sigma(G) = e(\sigma_1(G)) = e(G_0)$. Because G_0 is 0-valent, $e(G_0)$ must be 0-valent as well. Hence $\sigma(G)$ is 0-valent.

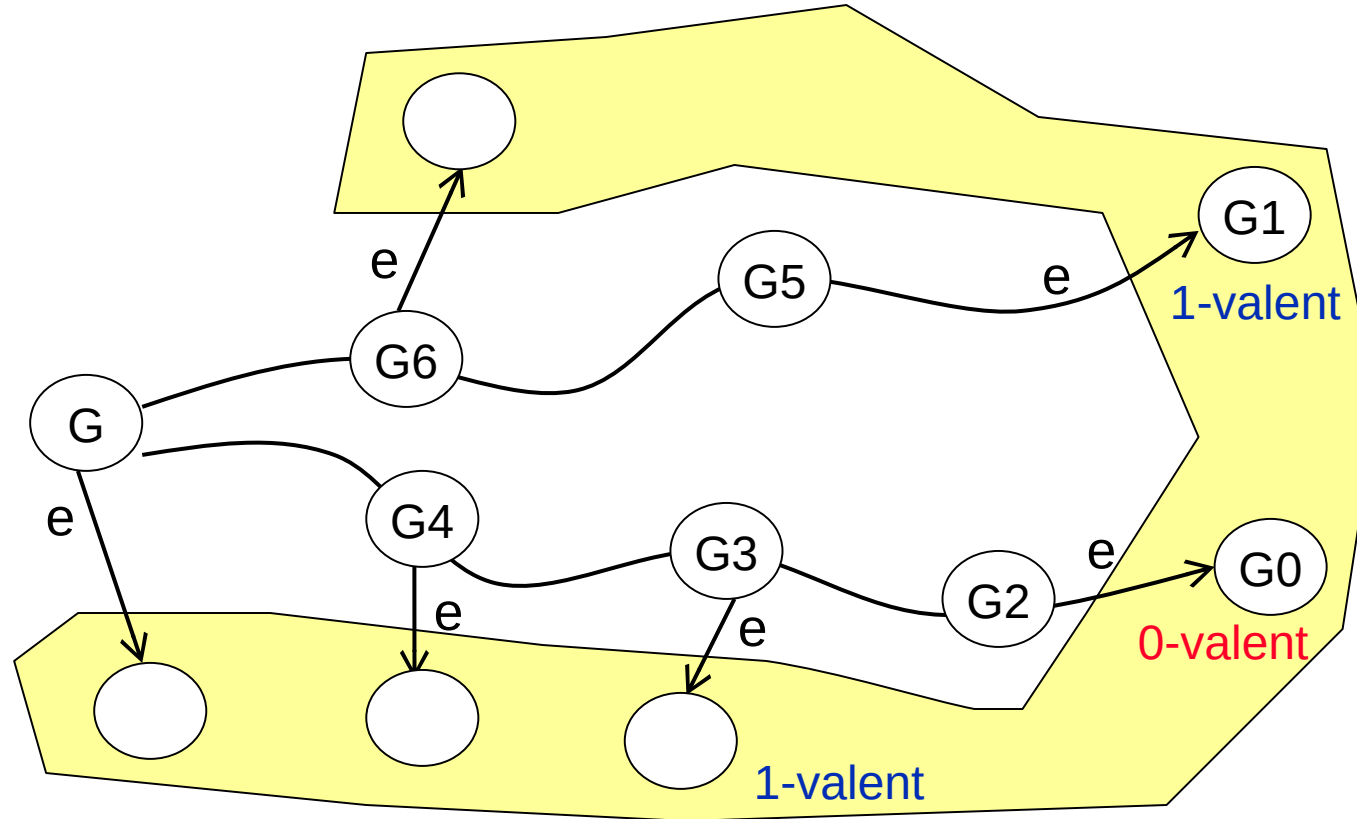


Proof for Lemma 4

- Claim 2: There must be a 0-valent state G_0 in V .
 - Proof: Consider the σ as defined in Claim 1 such that $\sigma(G)$ is 0-valent and σ contains e . Consider the prefix σ' of σ whose last event is e . (Hence prefix is the prefix from right to left.) Let $G_0 = \sigma'(G) \in V$.
Because V does not contain bivalent states and because the 0-valent state $\sigma(G)$ is reachable from G_0 , G_0 must be 0-valent.
- Claim 3: There must be a 1-valent state G_1 in V .

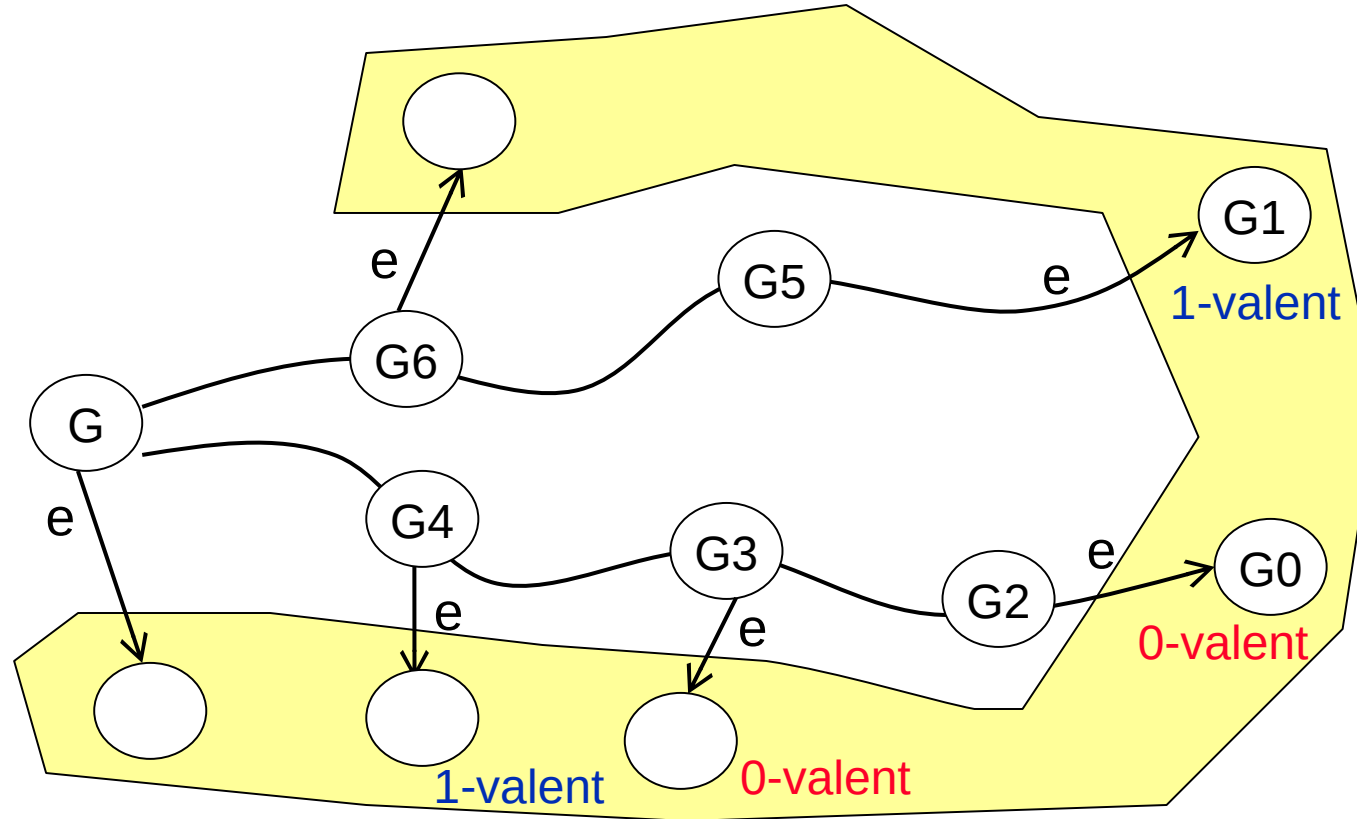
Proof for Lemma 4

- Claim 4: There must be $F0$ and $F1$ in W , such that $e(F0)$ is 0-valent, $e(F1)$ is 1-valent, and either $F1 = d(F0)$ or $F0 = d(F1)$.
 - Proof: Let $G0$ be a 0-valent state in V and $G1$ be a 1-valent state in V .



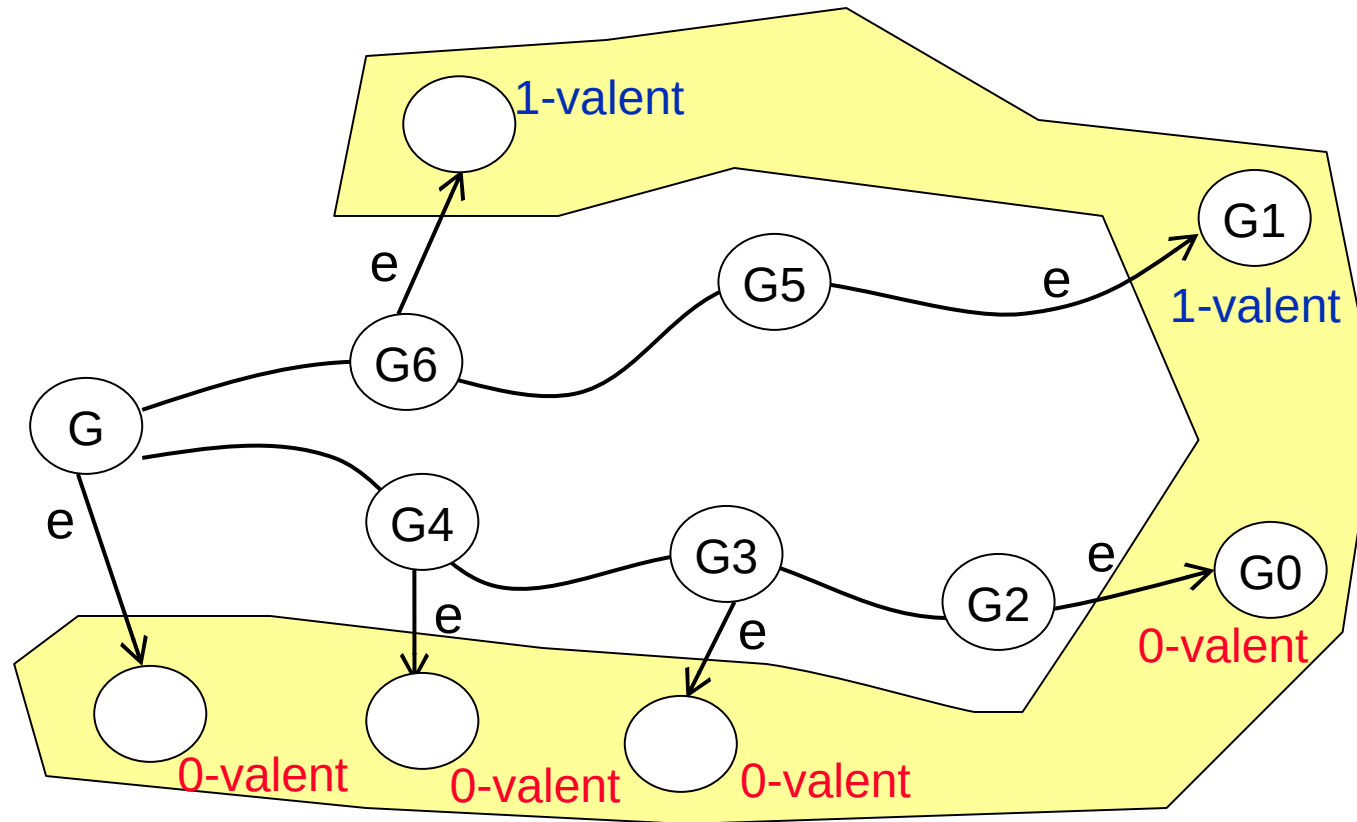
Proof for Claim 4

- Claim 4: There must be $F0$ and $F1$ in W , such that $e(F0)$ is 0-valent, $e(F1)$ is 1-valent, and either $F1 = d(F0)$ or $F0 = d(F1)$.
 - Proof: Let $G0$ be a 0-valent state in V and $G1$ be a 1-valent state in V .



Proof for Claim 4

- W.l.o.g., assume $e(G)$ is 0-valent. Suppose $G1 = e(\sigma^1(G))$. $|\sigma^1|$ must be at least 1 (otherwise $e(G)$ will be $G1$ and will be 1-valent).



Proof for Lemma 4

- Remaining proof for Lemma 4:
 - Consider $F0$ and $F1$ in W , such that $e(F0) = G0$ is 0-valent, $e(F1) = G1$ is 1-valent, and w.l.o.g. assume $F1 = d(F0)$. (By Claim 4)
 - e and d must occur on the same process p because otherwise $G1 = e(F1) = e(d(F0)) = d(G0)$ will have a decision of 0. (By Lemma 2)
 - Intuition: Whether the system decides on 0 or 1 entirely depends on the ordering of the two events e and d on the process p . Now if p is slow, other processes will have to decide without p telling them about the ordering...
 - Consider all possible executions starting from state $F0$. By termination requirement (and also to tolerate one process failure), there must be an execution where i) some process decides, and ii) process p does not execute any steps. Let the state immediately after some process decides be T where $T = \sigma(F0)$ and σ does not contain any step by p .
 - We have $e(T) = e(\sigma(F0)) = \sigma(e(F0)) = \sigma(G0)$ which is 0-valent (by Lemma 2)
 - We also have $e(d(T)) = e(d(\sigma(F0))) = \sigma(e(d(F0))) = \sigma(e(F1)) = \sigma(G1)$ which is 1-valent (by Lemma 2).
 - But some process has already decided in T . Regardless of whether the decision is 0 or 1, agreement can be violated. Contradiction.

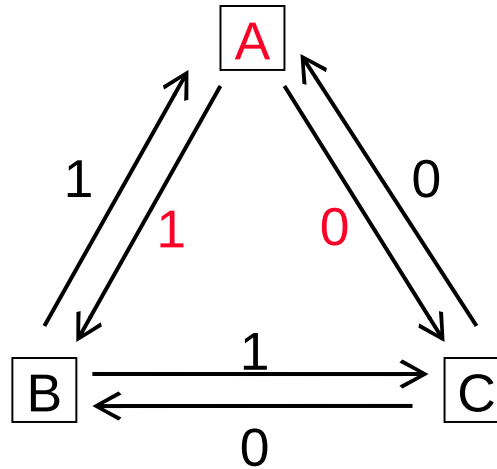
Distributed Consensus Version 4: Consensus with Node Byzantine Failures/Synchronous

- Failure model:
 - Nodes may experience **byzantine failures**
 - Communication channels are reliable
- Timing model:
 - Synchronous
- Goal:
 - **Termination**: All non-faulty nodes eventually decide
 - **Agreement**: All non-faulty nodes should decide on the same value
 - **Validity**: If all non-faulty nodes have the same initial input, that value should be the only possible decision value. Otherwise nodes are allowed to decide on anything (except that they still need to satisfy the Agreement requirement)

First (Unsuccessful) Attempt

- Simplified problem – 3 processes (A, B, C), 1 failure
 - Don't know which process fails
- Broadcast input to all other processes

B sees 1 from A, 1 from B, 0 from C $\Rightarrow$ B has to decide on 1, because C can be faulty



input: 1

input: 0

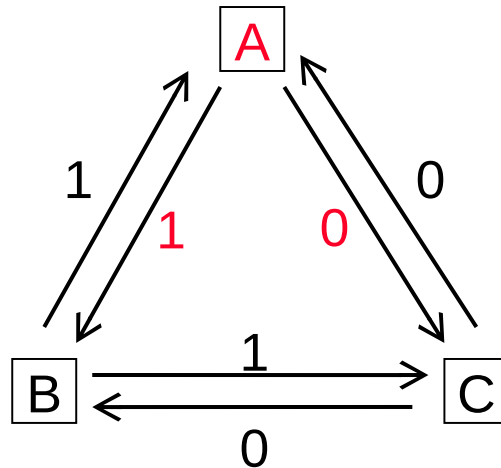
C sees 0 from A, 1 from B, 0 from C $\Rightarrow$ C has to decide on 0, because B can be faulty

Seems that B and C need to figure out that A is faulty in order for the protocol to work

Second (Unsuccessful) Attempt

- A second round (“C:1” means “C told me 1 in first round”)

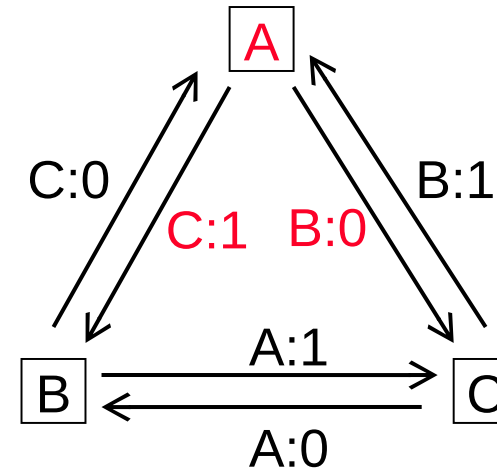
First Round



input: 1

input: 0

Second Round



B knows that some process is faulty;

But B still cannot figure out whether the faulty process is A or C

Byzantine Consensus Threshold

- Let n be the total number of processes, f be the number of possible byzantine failures
- Theorem: If $n \leq 3f$, then byzantine consensus problem (i.e., distributed consensus version 4) cannot be solved.
 - A non-trivial proof.
 - The earlier example does NOT constitute a proof (even for $f = 1$).

Byzantine Consensus Intuition

- We will develop a protocol for $n \geq 4f+1$
 - The definition of phase and round in the textbook is slightly confusing, we will use the definition as in the lecture notes
- Intuition:
 - A rotating coordinator paradigm – very useful!
 - Number the processes from 1 to n
 - Imagine a protocol with n phases – process i being the coordinator for phase i (only possible because we can define rounds!)
- Coordinator sends a proposal to all processes
 - Each phase has a **coordinator round** to do this
 - If coordinator is nonfaulty, all processes sees the proposal– consensus!
 - A phase is a **deciding phase** if the coordinator is nonfaulty

Byzantine Consensus Intuition

- With at most f failures and $f+1$ phases, at least one phase is a deciding phase
 - But what if the last phase has a faulty coordinator ?
 - Consensus decisions will be overruled!
- Avoiding a faulty coordinator to overrule the outcome of a deciding phase
 - After a deciding phase: All non-faulty processes have the same value
- Do not listen to the coordinator if
 - I see a lot of identical values from other processes
 - Each phase will also have a **all-to-all broadcast round**

n processes; at most f failures; $f+1$ phases; each phase has two rounds

Code for Process i :

$V[1..n] = 0$; $V[i] = \text{my input}$;

for ($k = 1$; $k \leq f+1$; $k++$) { // ($f+1$) phases

round for
all-to-all
broadcast

send $V[i]$ to all processes;
set $V[1..n]$ to be the n values received;
if (value x occurs $(> n/2)$ times in V) proposal = x ;
else proposal = 0;

coordinator
round

if ($k=i$) send proposal to all; // I am coordinator
receive coordinatorProposal from the coordinator

decide whether to
listen to coordinator

if (value y occurs $(> n/2 + f)$ times in V) $V[i] = y$;
else $V[i] = \text{coordinatorProposal}$;

}

decide on $V[i]$;

- Lemma 1: If all non-faulty processes P_i have $V[i] = y$ at the beginning of phase k , then this remains true at the end of phase k .

```
for (k = 1; k ≤ f+1; k++) { // (f+1) phases
    send V[i] to all processes;
    set V[1..n] to be the n values received;
    if (value x occurs (> n/2) times in V) proposal = x;
    else proposal = 0;

    if (k==i) send proposal to all; // I am coordinator
    receive coordinatorProposal from the coordinator

    if (value y occurs (> n/2 + f) times in V) V[i] = y;
    else V[i] = coordinatorProposal;
}
```

- Lemma 2: If the coordinator in phase k is nonfaulty, then all nonfaulty processes P_i have the same $V[i]$ at the end of phase k .

```
for (k = 1; k ≤ f+1; k++) { // (f+1) phases
    send  $V[i]$  to all processes;
    set  $V[1..n]$  to be the  $n$  values received;
    if (value  $x$  occurs ( $> n/2$ ) times in  $V$ ) proposal =  $x$ ;
    else proposal = 0;

    if (k==i) send proposal to all; // I am coordinator
    receive coordinatorProposal from the coordinator

    if (value  $y$  occurs ( $> n/2 + f$ ) times in  $V$ )  $V[i] = y$ ;
    else  $V[i] = \text{coordinatorProposal}$ ;
}
```

- Case 1: Coordinator has proposal = x; (x must be unique on coordinator)
 - On coordinator: x appears ($>n/2$) times in V $\Rightarrow$ ($>n/2-f$) must be from nonfaulty processes
 - On any other process: x appears ($>n/2-f$) times in V $\Rightarrow$ Impossible for y to appear ($>n/2+f$) times in V

```
for (k = 1; k ≤ f+1; k++) { // (f+1) phases
    send V[i] to all processes;
    set V[1..n] to be the n values received;
    if (value x occurs ( $> n/2$ ) times in V) proposal = x;
    else proposal = 0;

    if (k==i) send proposal to all; // I am coordinator
    receive coordinatorProposal from the coordinator

    if (value y occurs ( $> n/2 + f$ ) times in V) V[i] = y;
    else V[i] = coordinatorProposal; }
```

- Case 2: Coordinator has proposal = 0;
 - On coordinator: no value x appears ($>n/2$) times in V
 - On any other process: Impossible for y to appear ($>n/2+f$) times in V

```
for (k = 1; k ≤ f+1; k++) { // (f+1) phases
    send V[i] to all processes;
    set V[1..n] to be the n values received;
    if (value x occurs ( $> n/2$ ) times in V) proposal = x;
    else proposal = 0;

    if (k==i) send proposal to all; // I am coordinator
    receive coordinatorProposal from the coordinator

    if (value y occurs ( $> n/2 + f$ ) times in V) V[i] = y;
    else V[i] = coordinatorProposal;
}
```

Correctness Summary

- Lemma 1: If all nonfaulty processes P_i have $V[i] = y$ at the beginning of phase k , then this remains true at the end of phase k .
- Lemma 2: If the coordinator in phase k is nonfaulty, then all nonfaulty processes P_i have the same $V[i]$ at the end of phase k .
- Termination: Obvious ($f+1$ phases).
- Validity: Follows from Lemma 1.
- Agreement:
 - With $f+1$ phases, at least one of them is a deciding phase
 - (From Lemma 2) Immediately after the deciding phase, all nonfaulty processes P_i have the same $V[i]$
 - (From Lemma 1) In following phases, $V[i]$ on nonfaulty processes P_i does not change

Summary

<u>Failure Model and Timing Model</u>	<u>Consensus Protocol</u>
Ver 0: No node or link failures	Trivial – all-to-all broadcast
Ver 1: Node crash failures; Channels are reliable; Synchronous;	$(f+1)$ -round protocol can tolerate f crash failures
Ver 2: No node failures; Channels may drop messages (the coordinated attack problem)	Impossible without error Randomized algorithm with $1/r$ error prob
Ver 3: Node crash failures; Channels are reliable; Asynchronous;	Impossible (the FLP theorem)
Ver 4: Node Byzantine failures; Channels are reliable; Synchronous; (the Byzantine Generals problem)	If $n \leq 3f$, impossible. If $n \geq 4f + 1$, we have a $(2f+2)$ -round protocol. How about $3f+1 \leq n \leq 4f$?

Homework Assignment

- Why does the following algorithm not work for consensus under FLP assumptions? Give a scenario under which the algorithm fails. It is common knowledge that there are six processes in the system numbered P0 to P5. The algorithm is as follows: Every process sends its input bit to all processes (including itself) and waits for five messages. Every process decides on the majority of the five bits received.

Homework Assignment

- *Atomic broadcast* requires the following properties.
 - *Validity*: If the sender is correct and broadcasts a message m , then all correct processes eventually deliver m .
 - *Agreement*: If a correct process delivers a message m , then all correct processes deliver m .
 - *Integrity*: For any message m , q receives m from p at most once and only if p sent m to q .
 - *Order*: All correct processes receive all broadcast messages in the same order.

Show that atomic broadcast is impossible to solve in asynchronous systems.

- Homework due a week from today